Project Titan: Cart Super Add-On (CSAO) Rail

End-to-End Intelligent Recommendation Architecture

Team Name: KIINGSS

Team Members:
Aryaman Pathak
Hemant Gupta
Shreyas Biradar

Date: March 1, 2026

Abstract

This document presents a comprehensive technical blueprint for a real-time, context-aware recommendation engine designed to optimize Cart-to-Order ratios and Average Order Value (AOV). The solution utilizes a hybrid AI architecture combining **GraphSAGE** for retrieval, **BERT4Rec** for sequential context, and **DLRM** for precise ranking. It integrates Generative AI (LLMs) for semantic reasoning, Computer Vision (Swin Transformers) for visual complementarity, and Siamese Networks for item similarity. The system addresses complex business constraints (Cold Start, Price Anchoring, Dynamic Meal Completion) and scales to millions of daily requests with sub-200ms latency using a microservices architecture, vector databases (Milvus), and aggressive caching strategies.

Contents

1 Introduction & Problem Formulation **3**
1.1 Problem Statement . . . 3
1.2 Mathematical Formulation . . . 3

2 Data Strategy & Feature Engineering **4**
2.1 Synthetic Data Generation Strategy . . . 4
2.1.1 Data Structure & Distribution . . . 4
2.1.2 Data Visualization & Validation . . . 4
2.2 Feature Engineering Pipeline (SDE + AIML Perspective) . . . 4
2.2.1 Contextual Features (Real-Time) . . . 4
2.2.2 User & Historical Features . . . 5
2.2.3 Cart Composition Logic (Math) . . . 5
2.2.4 Data Preprocessing . . . 5

3 The AI Core: Multi-Stage Cascade **6**
3.1 System Overview . . . 6
3.2 Stage 1: Retrieval (GraphSAGE, Siamese, Milvus) . . . 6
3.2.1 Graph Neural Networks (GraphSAGE) . . . 6
3.2.2 Milvus Vector DB (Sub-10ms) . . . 6
3.3 Stage 2: Sequential Modeling (BERT4Rec) . . . 7
3.3.1 Why BERT4Rec over RNN/LSTM? . . . 7
3.4 Stage 3: Ranking (DLRM) . . . 7
3.5 Stage 4: Business Logic Re-Ranking . . . 7
3.5.1 Price Anchoring Logic . . . 7
3.5.2 Hyper-Local Geo-Specifics . . . 7
3.5.3 Proximity Logistics . . . 8
3.5.4 Rejection Feedback . . . 8

4 Advanced AI: GenAI, Vision & NLP **9**
4.1 GenAI LLM Integration . . . 9
4.1.1 RAG Pipeline (SDE + AIML View) . . . 9
4.2 Computer Vision: Appetite Appeal . . . 9
4.2.1 Swin Transformers YOLO . . . 9
4.3 NLP Text Analysis . . . 9

5 System Design Production Readiness **10**
5.1 Architecture Latency Budget ($<$ 200ms) . . . 10
5.2 Scalability Tactics . . . 10
5.3 Cold Start Strategy . . . 10

6 Evaluation Framework **11**
6.1 Model Performance Metrics 11
6.2 Business Impact Metrics 11
6.3 Operational Metrics 11

7 Implementation Plan **12**
7.1 Phase 1: Foundations (Weeks 1-4) 12
7.2 Phase 2: The Intelligence (Weeks 5-8) 12
7.3 Phase 3: Integration (Weeks 9-12) 12
7.4 Phase 4: Advanced Tuning (Weeks 13+) 12

8 Conclusion **13**

Chapter 1

Introduction & Problem Formulation

1.1 Problem Statement

Customers often create suboptimal carts, missing complementary items (e.g., ordering Biryani without Salan/Raita). The **CSAO Rail** aims to bridge this gap by intelligently suggesting add-ons.

The core challenges are:

- **Dynamic Context:** If a user adds *Subzi*, recommend *Roti*. If they add *Roti*, stop recommending breads and suggest *Desserts*.
- **Latency:** All computations must occur within **200-300ms**.
- **Price Anchoring:** If AOV is 200 and Cart is 190, do not recommend a 150 item.
- **Hyper-Local:** Recommend *Petha* in Agra, *Panchranga Achar* in Panipat.
- **Cross-Platform:** Leveraging Zomato + Blinkit synergy (e.g., Cake → Candles).

1.2 Mathematical Formulation

We define the problem as a **Sequential Context-Aware Ranking Optimization**.

Let U be the set of users, I be the set of items. At time t, for user u, the cart state is a sequence $S_u^t = \{i_1, i_2, ..., i_k\}$. We aim to select a subset of items $R \subset I$ such that we maximize the conditional probability of acceptance:

$$R^* = \underset{R,|R|=K}{\arg\max} \sum_{r \in R} P(acceptance|r, S_u^t, Context, History_u) \tag{1.1}$$

Subject to constraints:

- $Price(r) \leq \alpha \cdot \text{Value}(S_u^t)$ (Price Anchor, typically $\alpha = 0.4$)
- $Distance(Restaurant_r, User) \leq$ Threshold (Logistics)
- $Rating(r) \geq 4.0$ (Quality Constraint)

Chapter 2

Data Strategy & Feature Engineering

2.1 Synthetic Data Generation Strategy

To simulate the "messy" reality of food delivery, we employ a probabilistic generation framework.

2.1.1 Data Structure & Distribution

Entity	Attributes	Distribution Logic
Users	UserID, Age, Gender, City_Tier, Veg_Score, Price_Sensitivity	**Pareto Principle:** 20% of users generate 80% of orders. Gender split 60/40. Age groups weighted 18-35 (60%).
Restaurants	RestID, Cuisine, GPS, Rating (3.5-5.0), Avg_Prep_Time	**Clustered:** Restaurants generated in dense clusters (food hubs) vs sparse areas.
Items	ItemID, Name, Price, Is_Veg, Category, Image_URL	**Long Tail:** few "Head" items (Biryani) and many "Tail" items.
Interactions	SessionID, Timestamp, Event (View/Add/Buy/Reject), Cart_State	**Markov Chains:** Simulate meal progression (Starter → Main → Dessert).

2.1.2 Data Visualization & Validation

- **Sparsity Check:** Ensure Matrix Density $< 1\%$ (Real world is sparse).
- **Temporal Heatmaps:** Validate peaks at 1 PM (Lunch) and 8 PM (Dinner).
- **Price Distribution:** Log-normal distribution to mimic menu pricing.

2.2 Feature Engineering Pipeline (SDE + AIML Perspective)

We use a **Lambda Architecture** (Batch + Speed Layer).

2.2.1 Contextual Features (Real-Time)

- **Time Cyclical Encoding:**

$$T_{sin} = \sin(\frac{2\pi \cdot hour}{24}), \quad T_{cos} = \cos(\frac{2\pi \cdot hour}{24}) \tag{2.1}$$

- **Weather Integration:** Call OpenWeatherMap API. Map ‘Rain‘ → ‘Craving: Pakoda/Tea‘.
- **Occasion Guesser:**
  - If (Time > 10PM) & (Cart Size < 2) → "Late Night Craving".
  - If (Day = Fri/Sat) & (Cart Size > 5) → "Party/Family".
- **Festival Logic:** Check Calendar API. If ‘Date == Diwali‘, boost ‘Category: Sweets‘.

2.2.2 User & Historical Features

- **Demographics:** Age Group (18-24, 25-34), Gender (M/F).
- **Packaging Preference:** If user previously bought "Ketchup (Glass Bottle)", recommend "Glass" variants. If "Tetrapak", recommend "Tetrapak".
- **Past Order Logic:** If User ordered ‘Paneer Butter Masala‘ from ‘Rest A‘ last week, and adds ‘Naan‘ now, boost ‘Rest A‘.

2.2.3 Cart Composition Logic (Math)

- **Incomplete Meal Detection:**

$$Score_{meal} = w_1 \cdot I_{main} + w_2 \cdot I_{bread} + w_3 \cdot I_{drink} + w_4 \cdot I_{dessert} \tag{2.2}$$

  If $Score_{meal} < Threshold$, recommend missing category with highest weight.

- **Quantity Scaling:**

$$Rec_Size = \begin{cases} 500ml & \text{if } Cart_Items \leq 2 \\ 2Litre & \text{if } Cart_Items \geq 4 \end{cases} \tag{2.3}$$

2.2.4 Data Preprocessing

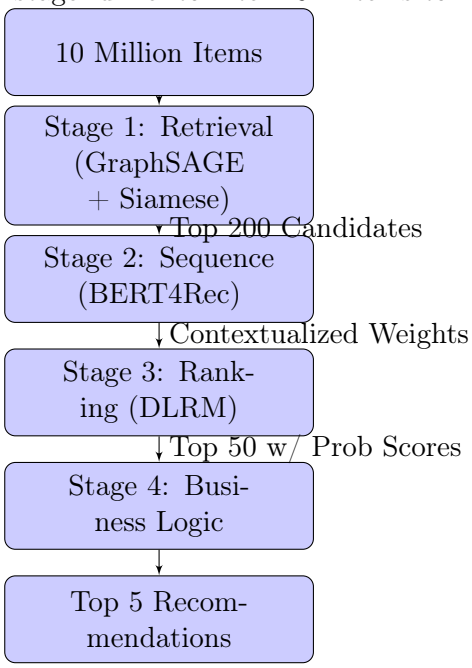
- **Cleaning:** Remove sessions with < 2 clicks.
- **Missing Values:** Impute Rating with Neighborhood Median.
- **Outliers:** Cap Price at 99th percentile.
- **Augmentation:** Random Item Dropout (Masking) during training to make model robust.

Chapter 3

The AI Core: Multi-Stage Cascade

3.1 System Overview

A 4-stage funnel to filter 10M items to Top 5 in $< 200ms$.



3.2 Stage 1: Retrieval (GraphSAGE, Siamese, Milvus)

3.2.1 Graph Neural Networks (GraphSAGE)

We build a Heterogeneous Graph (Users, Items, Restaurants).

- **Why?** Captures "Inductive" learning. Even if an item is new, if it's connected to a popular restaurant, it gets a good embedding.
- **Siamese Networks:** Used *before* the graph construction to compute Item-Item similarity based on Image + Text.

$$Loss = \max(0, m + D(A, P) - D(A, N)) \quad \text{(Triplet Loss)} \tag{3.1}$$

3.2.2 Milvus Vector DB (Sub-10ms)

- **Why Milvus?** Purpose-built for vectors, unlike generic SQL.

- **HNSW Algorithm:** Hierarchical Navigable Small World graphs. It creates layers of graphs. Search starts at top (fast, coarse) and drills down. Complexity $O(\log N)$.

3.3 Stage 2: Sequential Modeling (BERT4Rec)

3.3.1 Why BERT4Rec over RNN/LSTM?

- **RNNs:** Process left-to-right. Good for time-series, bad for carts where order is fluid.
- **BERT (Transformer):** Uses **Self-Attention**.
- **Dynamic Behavior:**
  - Input: '[Subzi, Roti]'. Model predicts '[Drink]'.
  - User removes 'Roti'. Input: '[Subzi]'.
  - Attention Mechanism instantly re-calculates. The connection between 'Subzi' and 'Bread' is strong.
  - Model predicts '[Naan/Kulcha]'.

3.4 Stage 3: Ranking (DLRM)

Facebook's Deep Learning Recommendation Model.

- **Embeddings:**
  - **User:** ID (64-dim) + Age/Gender (8-dim).
  - **Item:** ID (64-dim) + Visual (128-dim).
- **Interaction:** Dot Product to capture feature crosses.
- **Loss Function: Focal Loss**.

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \tag{3.2}$$

  Helps the model focus on hard-to-predict examples (rare adds).

3.5 Stage 4: Business Logic Re-Ranking

This layer applies hard constraints *after* the AI models.

3.5.1 Price Anchoring Logic

$$\text{Threshold} = \max(50, \text{CurrentCartValue} \times 0.4) \tag{3.3}$$

If Cart = 190, Threshold = 76. Filter out items > 76.

3.5.2 Hyper-Local Geo-Specifics

- **Logic:** Dictionary Map based on Geohash.
- *Agra:* Boost 'Petha'. *Panipat:* Boost 'Panchranga Achar'. *Assam:* Boost 'Tea'.
- **Popularity:** Suggest "Trending in your Area" (Top 5 items in last hour in 2km radius).

3.5.3 Proximity Logistics

- **Restaurant B Recommender:**
- If User orders from ‘Rest A‘.
- Check Google Routes API. Is ‘Rest B‘ within 500m of route ‘Rest A -> User‘?
- If Yes, show items from ‘Rest B‘ (Driver can batch orders).

3.5.4 Rejection Feedback

- **UI:** User clicks "Don’t show this".
- **Backend:** Add ‘ItemID‘ to Redis Bloom Filter for User.
- **Effect:** Item is masked for 7 days.

Chapter 4

Advanced AI: GenAI, Vision & NLP

4.1 GenAI LLM Integration

4.1.1 RAG Pipeline (SDE + AIML View)

- **Scenario:** "I added Cake. What else?"
- **Retrieval:**
  - Keyword Search: "Birthday", "Party".
  - Vector Search: ‘Embedding("Cake")‘ similarity with Blinkit items.
- **Generation (Llama 3):**
  - **Fine-Tuning: QLoRA** (Quantized Low Rank Adaptation) on food pairings.
  - **Sampling: Nucleus Sampling (Top-p = 0.9)**. This ensures diversity (doesn't just say "Candles" every time, might say "Balloons").

4.2 Computer Vision: Appetite Appeal

4.2.1 Swin Transformers YOLO

- **YOLO v8:** Object Detection. Detects "Green Dot" (Veg) or "Chili Icon" (Spicy) on packaging.
- **Swin Transformer:** Extracts visual embeddings (Texture, Color).
- **Logic:**
  - *Input:* Cart has "Red/Oily" images (Butter Chicken).
  - *Recommendation:* Find "Cool/White" embeddings (Lassi, Ice Cream) to visually balance the meal.

4.3 NLP Text Analysis

- **Technique:** BERT Embeddings on Item Descriptions.
- **Usage:** Mapping "Spicy" keywords to User Preference "High Spice Tolerance".

Chapter 5

System Design Production Readiness

5.1 Architecture Latency Budget ($<$ 200ms)

Component	**Tech Stack**	**Latency Budget**
Gateway	NGINX Load Balancer	10ms
Feature Store	Feast + Redis Cluster	15ms
Retrieval	Milvus (HNSW Index)	35ms
Ranking Model	NVIDIA Triton (GPU)	80ms
Business Logic	GoLang Rule Engine	20ms
Network Overhead	gRPC (Protobuf)	40ms
Total		**200ms**

5.2 Scalability Tactics

- **Sharding:** Database sharded by 'User ID
- **Master-Slave:** PostgreSQL for Order History. 1 Master (Write), 5 Slaves (Read).
- **Caching:**
  - **L1 (Local):** Top 50 Global Items.
  - **L2 (Redis):** User Session Vectors.
- **Circuit Breaker:** If Model Latency $>$ 250ms, trip breaker and serve "Popular Items" from Redis (Fallback).
- **Rollbacks:** Blue/Green Deployment.

5.3 Cold Start Strategy

- **New User:** Ask 3 questions (Veg/Non-Veg, Spice Level, Sweet Tooth?). Use "Popular in Geohash".
- **New Item:** Use **Siamese Network** to link it to similar existing items via Image/Description.

Chapter 6

Evaluation Framework

6.1 Model Performance Metrics

Metric	Definition & Usage
AUC (ROC)	Area Under Curve. Measures ability to distinguish between a "bought" add-on and an ignored one. Target: > 0.8.
NDCG@K	Normalized Discounted Cumulative Gain. Rewards correct items appearing at the *top* of the list.
Precision@K	$\frac{\text{Relevant Items in Top K}}{K}$. High precision prevents user annoyance.
Recall@K	$\frac{\text{Relevant Items in Top K}}{\text{Total Relevant Items}}$. Ensures we don't miss obvious add-ons.

6.2 Business Impact Metrics

- **AOV Lift:** (Avg Order Value Treatment - Control). The North Star metric.
- **CSAO Attach Rate:** % of orders containing ≥ 1 recommended item.
- **Cart-to-Order (C2O) Ratio:** Monitoring if recommendations cause friction/abandonment.
- **CTR:** Click-Through Rate on recommendation rail.

6.3 Operational Metrics

- **Inference Latency:** p95 and p99.
- **Feature Freshness:** Time lag between "Add to Cart" and "Feature Available". Target $< 1s$.
- **Coverage:** % of carts receiving valid recommendations.

Chapter 7

Implementation Plan

7.1 Phase 1: Foundations (Weeks 1-4)

- **Infrastructure:** Setup AWS EKS, S3, Kafka.
- **Data:** Ingest historical logs. Cleanse data (Missing values, Outliers).
- **Baseline:** Implement "Popular in GeoHash" heuristic.

7.2 Phase 2: The Intelligence (Weeks 5-8)

- **Training:** Train GraphSAGE and BERT4Rec on GPU Cluster.
- **Vector DB:** Setup Milvus and index 1M items using HNSW.
- **Validation:** Offline evaluation using NDCG on holdout set.

7.3 Phase 3: Integration (Weeks 9-12)

- **Serving:** Deploy models to NVIDIA Triton.
- **Logic:** Implement Price Anchoring and Proximity rules in GoLang.
- **Shadow Mode:** Run model on live traffic without showing results to users.

7.4 Phase 4: Advanced Tuning (Weeks 13+)

- **GenAI:** Integrate Llama 3 for RAG explanations.
- **A/B Testing:** Roll out to 5% users. Monitor AOV Lift.
- **Optimization:** Quantize models to INT8 for speed.

Chapter 8

Conclusion

Team **KIINGSS** presents a robust, end-to-end solution for the CSAO Rail. We move beyond simple statistical correlation to true **Contextual Understanding** using **BERT4Rec** (Sequence), **GraphSAGE** (Connectivity), and **GenAI** (Semantics).

Our architecture explicitly addresses:

- **Latency:** Through Milvus, HNSW, and TensorRT Quantization.
- **Business constraints:** Price anchoring, Logistics optimization, and Rejection feedback.
- **User Experience:** Visual complementarity and "Appetite Appeal".

By combining rigorous SDE principles (Microservices, Caching, Sharding) with cutting-edge AIML (Transformers, GNNs, LLMs), this system is poised to deliver a measurable lift in AOV while delighting customers with "magic" recommendations.